

Домашнее задание 2

Лиза Фоменко

Задание 1

Найдите Θ -асимптотику функции $f(n) = \sum_{k=0}^n \binom{n}{k}$

Докажем по индукции с использованием следующих фактов:

1. $\binom{n}{0} = \binom{n+1}{0} = 1$
2. $\binom{n}{n} = \binom{n+1}{n+1} = 1$
3. $\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k}$; доказательство этого факта: $\binom{n}{k} + \binom{n}{k-1} = \frac{n!}{k!(n-k)!} + \frac{n!}{(k-1)!(n-k+1)!} = \frac{n!}{(k-1)!(n-k)!} * \left(\frac{1}{k} + \frac{1}{n-k+1}\right) = \frac{n!}{(k-1)!(n-k)!} * \frac{n-k+1+k}{k*(n-k+1)} = \frac{n!}{(k-1)!(n-k)!} * \frac{n+1}{k*(n-k+1)} = \frac{(n+1)!}{k!(n-k+1)!} = \binom{n+1}{k}$

Далее воспользуемся индукцией по n .

База индукции: $2^1 = \binom{1}{0} + \binom{1}{1} = 1 + 1 = 2$

Предположение индукции: $2^n = \sum_{k=0}^n \binom{n}{k}$

Шаг индукции: $2^{n+1} = 2 * 2^n = 2 * \sum_{k=0}^n \binom{n}{k} = 2 * \left(\binom{n}{0} + \binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n-2} + \binom{n}{n-1} + \binom{n}{n}\right) = 2 * \binom{n}{0} + 2 * \binom{n}{1} + 2 * \binom{n}{2} + \dots + 2 * \binom{n}{n-2} + 2 * \binom{n}{n-1} + 2 * \binom{n}{n} = \binom{n+1}{0} + \left(\binom{n}{0} + \binom{n}{1}\right) + \left(\binom{n}{1} + \binom{n}{2}\right) + \dots + \left(\binom{n}{n-2} + \binom{n}{n-1}\right) + \left(\binom{n}{n-1} + \binom{n}{n}\right) + \binom{n+1}{n} = \binom{n+1}{0} + \binom{n+1}{1} + \binom{n+1}{2} + \dots + \binom{n+1}{n-2} + \binom{n+1}{n-1} + \binom{n+1}{n} = \sum_{k=0}^{n+1} \binom{n+1}{k}$

По мат индукции доказали, что $2^n = \sum_{k=0}^n \binom{n}{k}$. Отсюда воспользуемся так называемым "свойством рефлексивности" из Кормена: $f(n) = \Theta(f(n)) \Rightarrow \sum_{k=0}^n \binom{n}{k} = \Theta(2^n)$

Задание 2

Дан набор из n монет, одна из которых фальшивая и легче других, и чашечные весы, на которые можно положить две группы монет одинакового размера и узнать, какая из них тяжелее, или что они равны по весу. Придумайте алгоритм, доказжите его корректность и оцените асимптотику.

оформлю словесный алгоритм в виде псевдокода :)

while True:

if число монет в куче $n \bmod 3 == 0$:

 делим кучу на 3 равные части: куча1 = $n/3$, куча2 = $n/3$, куча3 = $n/3$ монет

elif число монет в куче $n \bmod 3 == 1$:

 делим кучу на 3 части: куча1 = $n//3$, куча2 = $n//3$ и куча3 = $n//3 + 1$ монет

else:

 делим кучу на 3 части: куча1 = $n//3 + 1$, куча2 = $n//3 + 1$, куча3 = $n//3$ монет

взвешиваем кучу1 и кучу2 (всегда равны по кол-ву монет):

if одинаковые по весу:

 куча = куча3

if число монет в куче == 1:

 return куча (нашли монетку)

else:

 куча = min(куча1, куча2)

if число монет в куче == 1:

 return куча (нашли монетку)

Корректность: на i -м шаге мы ищем $\min(kucha_{i1}, kucha_{i2}, kucha_{i3})$ за 1 сравнение, где $kucha_{ik}$ - одна из трех частей $kucha_i$, которую мы приняли на вход нашего i -го шага.

Инвариант: на i шаге $kucha_i = \min(kucha_{(i-1)1}, kucha_{(i-1)2}, kucha_{(i-1)3})$.

База индукции: на первом шаге мы делим исходную кучу на 3 части - $kucha_{11}, kucha_{12}$ и $kucha_{13}$. За одно взвешивание $kucha_1$ и $kucha_2$ с равным количеством монеток (важно для работы алгоритма!) получим, что либо они одинаковы $\Rightarrow$ легкой монетки в них нет $\Rightarrow$ она в 3, либо одна из них будет легче $\Rightarrow$ легкая монетка в более легкой из них. Тогда $kucha_2 = \min(kucha_{11}, kucha_{12}, kucha_{13})$.

Шаг индукции: $kucha_{i+1} = \min(kucha_{i1}, kucha_{i2}, kucha_{i3}) = \min(kucha_i/3, kucha_i/3, kucha_i/3) = \min(\min(kucha_{(i-1)1}, kucha_{(i-1)2}, kucha_{(i-1)3}/3), \min(kucha_{(i-1)1}, kucha_{(i-1)2}, kucha_{(i-1)3}/3), \min(kucha_{(i-1)1}, kucha_{(i-1)2}, kucha_{(i-1)3}/3))$ (здесь показано целочисленное деление для простоты, но зависит от случая :)

Асимптотика: На каждом шаге алгоритма мы:

1. делим кучу на 3 части $\Theta(1)$
2. взвешиваем 2 из них $\Theta(1)$
3. выбираем самую легкую кучу $\Theta(1)$

То есть каждый шаг алгоритма выполняется за $\Theta(1)$ (это, конечно, некоторого рода абстракция; скорее всего, в реализации кодом "взвешивание" выглядело бы как к-л сумма / лин операция, и ее нельзя было бы считать за $\Theta(1)$). Остается только лишь подсчитать, сколько шагов мы сделаем: с каждым шагом размер кучи уменьшается в 3, поэтому шагов будет порядка $\lceil \log_3 n \rceil$ (пока не дойдем до кучек размера 1), где n - общее количество монет. Тогда: $\lceil \log_3 n \rceil * \Theta(1) = \Theta(\log_3 n)$.

ответ: асимптотика алгоритма $\Theta(\log_3 n)$. Это оптимальный алгоритм.

Задание 3

Найдите нижнюю оценку для задачи поиска фальшивой монеты, не предполагая ничего про алгоритм. Нужно найти нижнюю оценку сложности именно для задачи, а не для какого-либо конкретного алгоритма, решающего ее.

Пусть у нас есть дерево, где узлы - сравнения (взвешивания), а листья - исходы всех взвешиваний (условно, i лист = i -я монета фальшивая). С каждого узла мы имеем 3 возможных исхода: $=$, $<$, $>$. Тогда наше дерево - полное M -арное, где $M=3$ (тернарное, кажется), а количество листьев = n . Тогда высота такого дерева составляет $\lceil \log_3 n \rceil$. Высота дерева в данном контексте - это количество взвешиваний, которые могут довести нас от изначальной кучи до фальшивой монеты (само значение высоты при известном количестве листьев использую без доказательства, но оно аналогично бинарному дереву, пример в Кормене).

Тогда для любой входной кучи из n монет мы можем за $\lceil \log_3 n \rceil$ взвешиваний дойти до любого(!) его листа.

Если же высота дерева будет меньше, чем $\lceil \log_3 n \rceil$, например $\lceil \log_3 n \rceil - 1$? Дерево остается тернарным, полным, и для него верно, что при высоте $\lceil \log_3 n \rceil - 1$ максимальное количество его листьев (= возможных исходов) будет равно $3^{\lceil \log_3 n \rceil - 1} = n/3$, то есть за такое количество взвешиваний у нас останется доступ только к $n/3$ возможным исходам: перемешав кучу, мы можем не попасть в нужный лист (нарушение детерминированности алгоритма). Поэтому нижняя оценка алгоритма составляет $\Omega(\lceil \log_3 n \rceil)$.

Мой алгоритм работает за $\Theta(\log_3 n)$, то является оптимальным.

Задание 4

Найдите $7^{13} \bmod 167$ с помощью быстрого возведения в степень. Нужно привести последовательность умножений и промежуточные результаты. $\bmod 167$ - это остаток от деления на 167.

Будем использовать быстрое возведение в степень за $\log_2 n$ и следующий факт: $(a*b) \bmod p = (a \bmod p) * (b \bmod p) \bmod p$.

1. $7 \bmod 167 = 7$
2. $7^2 \bmod 167 = (7 \bmod 167)^2 \bmod 167 = 7*7 \bmod 167 = 49$
3. $7^3 \bmod 167 = (7^2 \bmod 167) * (7 \bmod 167) \bmod 167 = (49 * 7) \bmod 167 = 343 \bmod 167 = 9$
4. $7^6 \bmod 167 = (7^3 \bmod 167)^2 \bmod 167 = 9*9 \bmod 167 = 81$

$$5. 7^{12} \bmod 167 = (7^6 \bmod 167)^2 \bmod 167 = 81 \cdot 81 \bmod 167 = 6561 \bmod 167 = 48$$

$$6. 7^{13} \bmod 167 = (7^{12} \bmod 167) \cdot (7 \bmod 167) \bmod 167 = (48 \cdot 7) \bmod 167 = 2$$

ответ: 2

Задание 5

Рассмотрим последовательность L_i , которая похожа на числа Фибоначчи, но в обратную сторону. Она будет начинаться с двух положительных целых чисел $L_0 = a$ и $L_1 = b \leq a$, а следующие элементы будут определяться как $L_{i+1} = \max(L_i, L_{i-1}) - \min(L_i, L_{i-1})$. Определим, сколько ненулевых элементов может быть в такой последовательности, если она начинается с двух чисел, запись большего из которых состоит из n бит. Как изменится ответ, если вместо вычитания брать остаток от деления нацело?

$$1) L_{i+1} = \max(L_i, L_{i-1}) - \min(L_i, L_{i-1})$$

Мне не очень понятен вопрос "сколько" элементов. Имеется в виду сколько различных элементов может быть в последовательности или сколько *всего* ее членов являются ненулевыми?

Рассмотрим оба варианта постановки вопроса. Начнем с вопроса *сколько всего* ненулевых членов в последовательности L_i , для этого рассмотрим следующий пример:

Пусть у нас конечное число ненулевых элементов в последовательности, тогда (так как она в целом убывающая) в какой-то момент, начиная с $L_k, \exists k \in \mathbb{N}$, что для $\forall n \geq k$ верно, что $L_n = 0$. Тогда рассмотрим L_{k+1} и L_k . Мы знаем, что $L_{k+1} = |L_k - L_{k-1}| = |0 - L_{k-1}| = |L_{k-1}| = 0 \Rightarrow L_{k-1} = 0$. Однако мы предположили, что последовательность 0 начинается только с L_k ее члена. Следуя такой логике, мы получим, что вся такая последовательность состоит только из 0.

Рассмотрим случай, когда L_k оказалось равно 0. Это значит, что $L_k = |L_{k-1} - L_{k-2}| = 0 \Rightarrow L_{k-1} = L_{k-2}$. Тогда два следующих члена: $L_{k+1} = |L_k - L_{k-1}| = |0 - L_{k-1}| = |L_{k-1}| = L_{k-2}$; $L_{k+2} = |L_{k+1} - L_k| = |L_{k-1} - 0| = |L_{k-1}| = |L_{k-1}| = L_{k-2}$. То есть когда мы получаем 0 в последовательности, она начинает выглядеть как $x, x, 0, x, x, 0, \dots$ и так бесконечно.

То есть L_i имеет бесконечное общее число ненулевых элементов. Разберемся с числом различных элементов в этой последовательности.

Мы знаем, что a состоит из n бит, то есть максимальное значение $a = 11\dots 1$ (n единиц). Также $a \geq b > 0$, то есть наша последовательность точно будет убывающей. Сколько всего существует n -битных чисел (то есть у нас n позиций, на каждой из которых может стоять либо 1, либо 0)? Ответ: 2^n чисел. Тогда попробуем подобрать пример, в котором встретятся все 2^n чисел.

1. $L_0 = a = 11\dots 11$, n единиц
2. $L_1 = b = 1 = 1$
3. $L_2 = |L_0 - L_1| = |a - b| = |11\dots 11 - 1| = 11\dots 10$
4. $L_3 = |L_1 - L_2| = |1 - 11\dots 10| = 11\dots 01$
5. $L_4 = |L_2 - L_3| = |11\dots 01 - 11\dots 10| = 1$
6. $L_5 = |L_3 - L_4| = |11\dots 01 - 1| = 11\dots 100$
7. $L_6 = |L_4 - L_5| = |1 - 11\dots 00| = 11\dots 011$
8. $L_7 = |L_5 - L_6| = |11\dots 011 - 11\dots 100| = 1$
9. etc, $L_{3k+1} = 1, L_{3k-1} - L_{3k} = 1$

Как видно из приведенных выше первых членов и их общих формул, составленная таким образом L_i будет содержать все 2^n чисел (ее конечная повторяющаяся бесконечная часть будет выглядеть как $1, 1, 0, 1, 1, 0, \dots$).

Итого ответ: разных ненулевых чисел - $2^n - 1$; всего ненулевых чисел - бесконечное количество (так как с k -л момента последовательность становится периодичной).

$$2) L_{i+1} = \max(L_i, L_{i-1}) \bmod \min(L_i, L_{i-1})$$

Наша последовательность прервется, когда $L_k = 0$ для какого-либо k (так как мы работаем с положительными числами, то 0 будет минимальным в паре с любым другим числом, и мы в последовательности будем пытаться искать остаток от деления L_{k-1} на $L_k = 0$, получим неопределенность). Тогда, дойдя

до первого 0 в последовательности, она завершится. При этом наша последовательность всегда будет конечна, так как частное от деления всегда меньше, чем делитель, а мы начинаем с целого числа a .

Здесь нам необходимо максимизировать число ненулевых элементов, то есть построить наиболее медленно убывающую последовательность: для этого необходимо получать наибольший возможный остаток, что (в свою очередь) будет при частном = 1.

Тогда рассмотрим L_{i-1}, L_i, L_{i+1} : зная, что частное = 1, запишем выражение для L_{i+1} : $L_{i+1} = L_i * 1 + L_{i-1} = L_i + L_{i-1} \Rightarrow$ мы получили последовательность Фибоначчи, с тем лишь небольшим отличием, что начало истинной послед. Фибоначчи выглядит как 0, 1, 1, 2, 3; а конец нашей последовательности - 3, 1, 0 (так любое число при делении на 1 дает остаток 0).

Наибольшая последовательность нужного вида $F_k, F_{k-1}, \dots, 3, 2, 1, 0$, где F_k - самое большое n -битное число Фибоначчи.

Мне, к сожалению, не удалось вывести общий вид числа Фибоначчи в двоичном коде, чтобы по данному n общей формулой вывести F_k (наибольшее n -битное число Фибоначчи). Замечу, что если по n научиться вычислять k , то длина такой последовательности будет $k - 2$ (так как получим последовательность ненулевых чисел от F_k до $F_2 = 1$, $F_1 = 1$ не используется (см выше), а $F_0 = 0$).